

毕业设计最终答辩

Rust异步驱动模块设计与实现

汇报人：黄昊颖 指导老师：向勇

目录

Contents

研究背景与问题

01

目标与主要工作

02

现有机制与切入点

03

设计思路与取舍

04

正确性与可用性

05

结论

06

后续工作设想

07



01.7

研究背景与问题

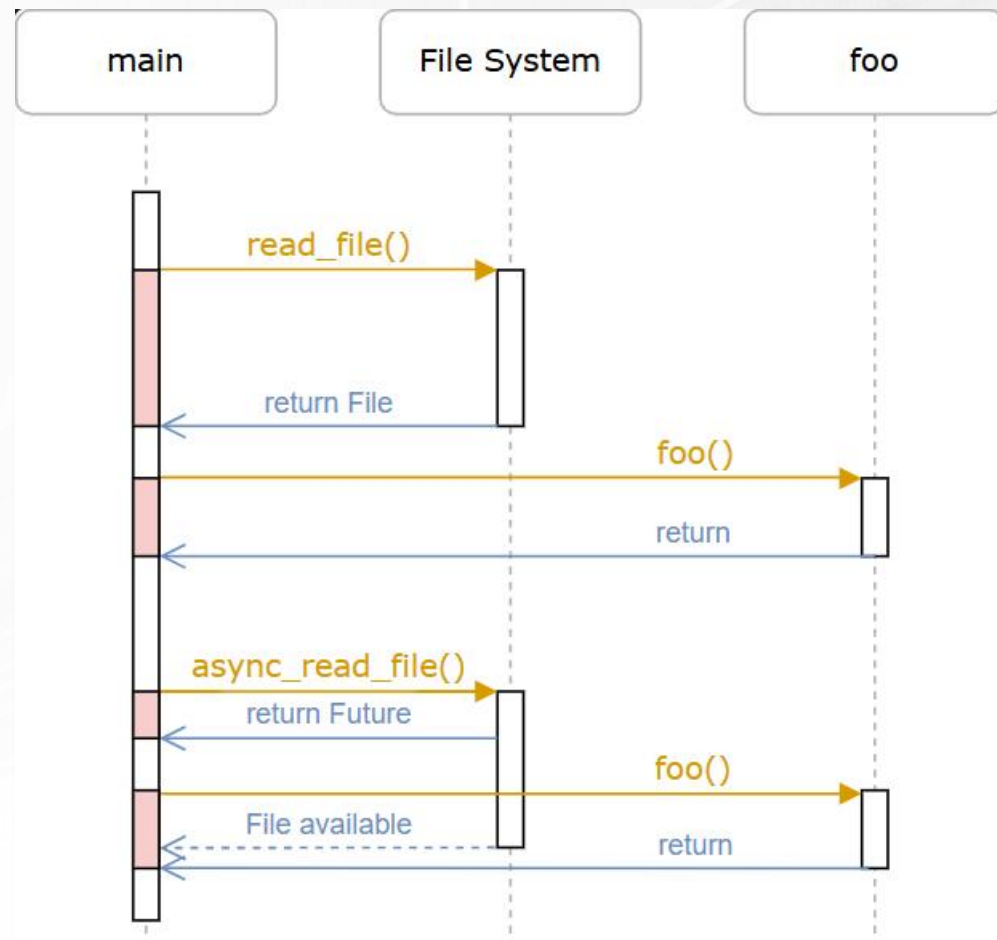
研究背景与问题

研究背景

- 1) 多核内核需要同时处理调度/中断/syscall
- 2) 传统busy-wait/同步阻塞模型问题
- 3) Rust Future: 异步状态机

研究问题

能否把 Rust 异步机制接入 rCore-N 内核
timer 路径



async 示例: <https://os.phil-opp.com/async-await/>

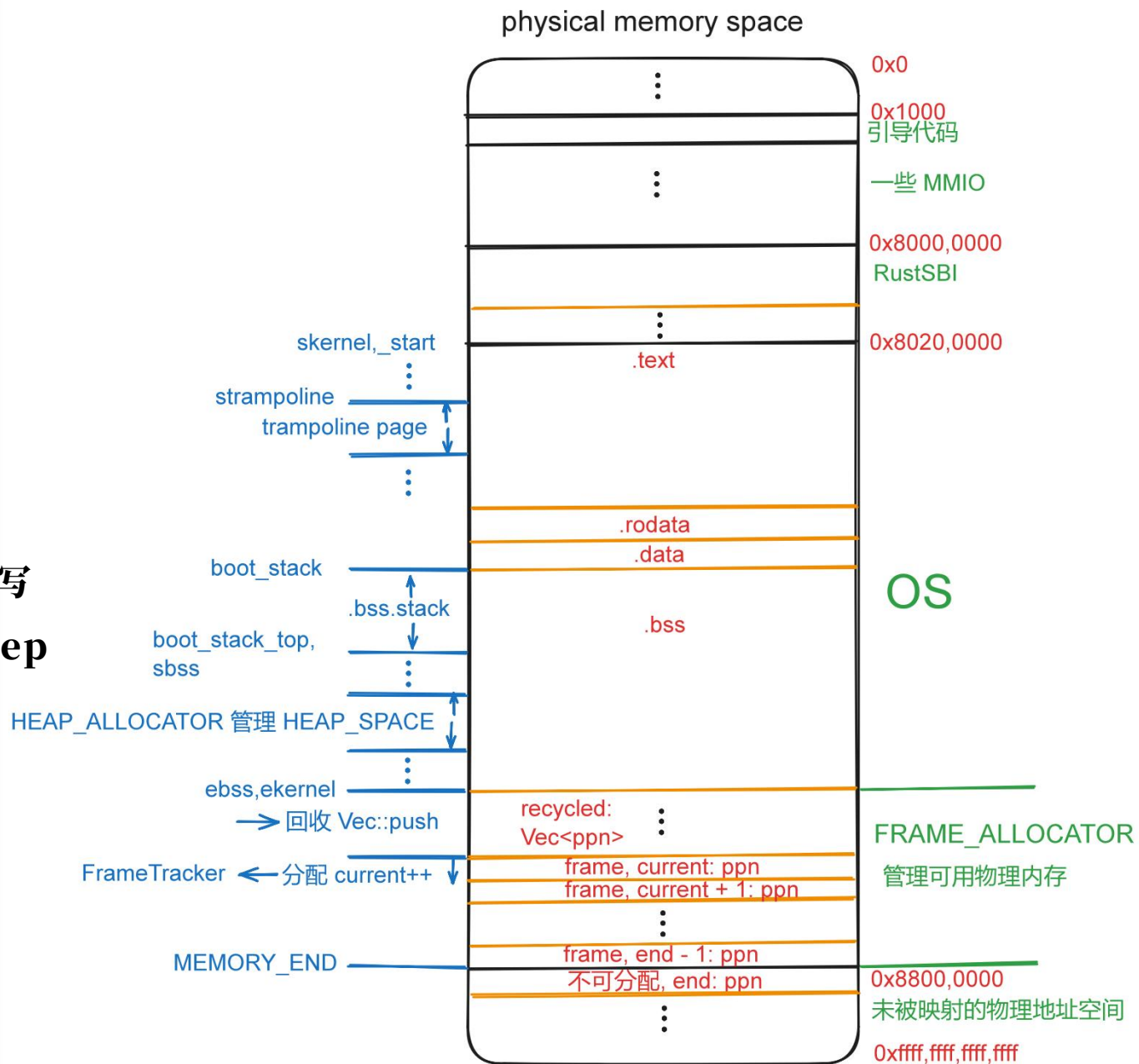
02.7

目标与主要工作

目标与主要工作

实验步骤

- 1) rCore-N 环境配置
- 2) rCore-N 异步 timer 开发
 - 2.1) 参考 embassy timer 实现
 - 2.2) 原 timer 逻辑 Rust Future 重写
 - 2.3) 实现基于异步 timer 的 sys_sleep
- 3) rCore-N 异步 timer 测试
 - 3.1) 可用性实验验证



rCore/QEMU 物理地址空间

03.7

现有机制与切入点

「现有机制与切入点

现有机制

1) rCore-2025S

- 1.1) 固定 10ms tick
- 1.2) 阻塞式 sys_sleep

2) rCore-N

- 2.1) 用户态中断 utimer: sys_set_timer()
- 2.2) 未实现 sys_sleep
- 2.3) 用户库 user_lib::sleep 是 busy-wait

切入点

在 rCore-N 内核态重新实现 sys_sleep

- 1) 基于 Rust Future

```
[DEBUG 0]: boot_stack 0x80534000 top 0x80574000
[DEBUG 0]: trying to add initproc
[DEBUG 0]: add_initproc
[DEBUG 0]: initproc added to task manager!
[hart 0]satp: 0x8000000000080775, sp: 0x80543ed0
[DEBUG 0]: [kernel 0] Start 1
[DEBUG 0]: [kernel 0] Start 2
[hart 1]satp: 0x8000000000080775, sp: 0x80553ed0
[hart 2]satp: 0x8000000000080775, sp: 0x80563ed0
[DEBUG 0]: [kernel 0] Start 3
[hart 1>Hello
[hart 2>Hello
[hart 0>Hello
[hart 3]satp: 0x8000000000080775, sp: 0x80573ed0
```

rCore-N 运行记录：内核输出

04.7

设计思路与取舍

设计思路与取舍

Embassy 对设计的启发

Embassy 学习：运行 Embassy 示例程序并追踪程序执行流

1) embassy-executor 状态 State 转移

2) embassy timer 设计

2.1) embassy-rp

2.2) 基于中断的 executor

3) embassy 主要数据结构之间的转换

各个数据结构的设计与访问并非孤立而是互相联系

设计思路与取舍

Embassy 对设计的启发

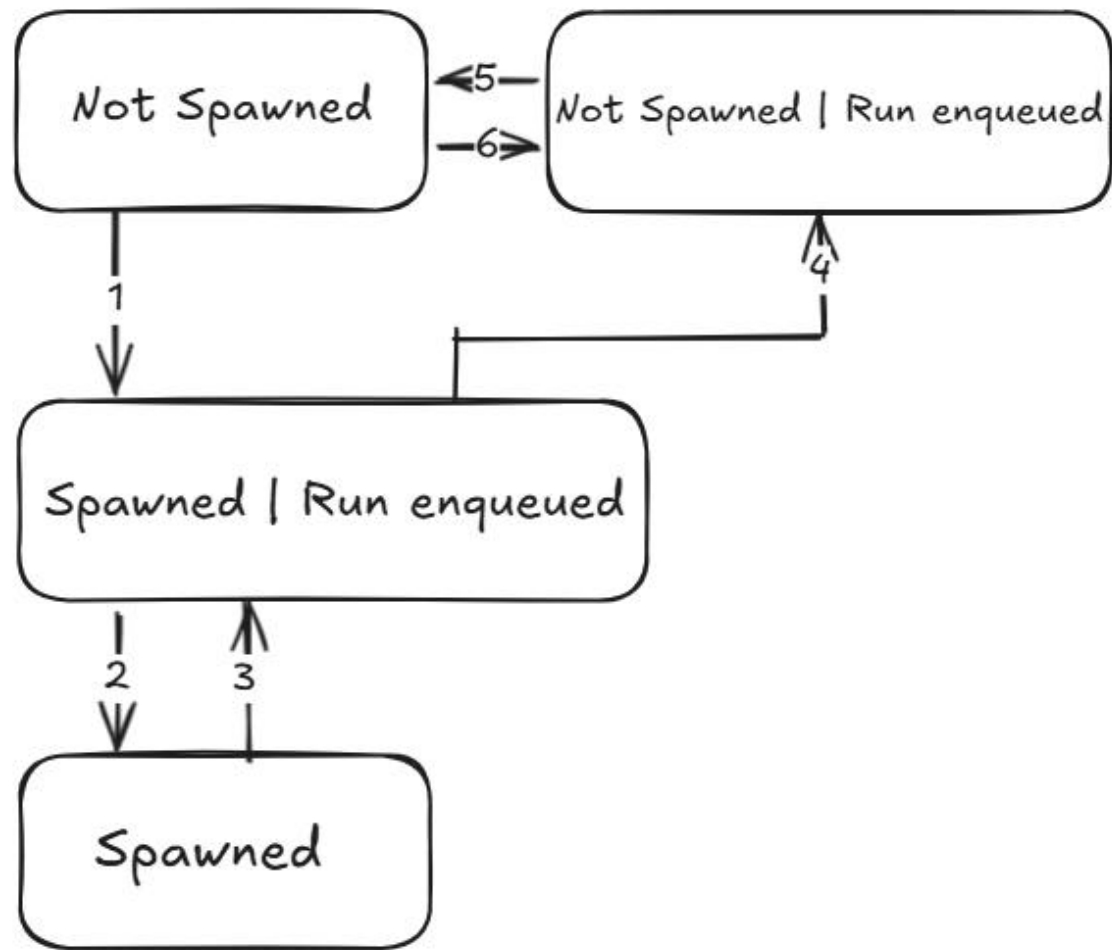
1) embassy-executor 状态 State 转移

1.1) **Not Spawned** 初始状态 TaskStorage 槽位为空没有正在运行的 future

1.2) **Spawned | Run enqueued** 任务 future 存在且在 RunQueue 中等待下次 poll() 执行 poll_fn

1.3) **Spawned** 任务 future 存在但不在 RunQueue 中所以不会马上被 poll

1.4) **Not spawned | Run enqueued** 此时 poll_fn 已经被换成空的 poll_exited() 了, 任务已经执行结束了但还在 RunQueue 中等下次 poll() 被清理



来自 embassy-executor/src/raw/mod.rs

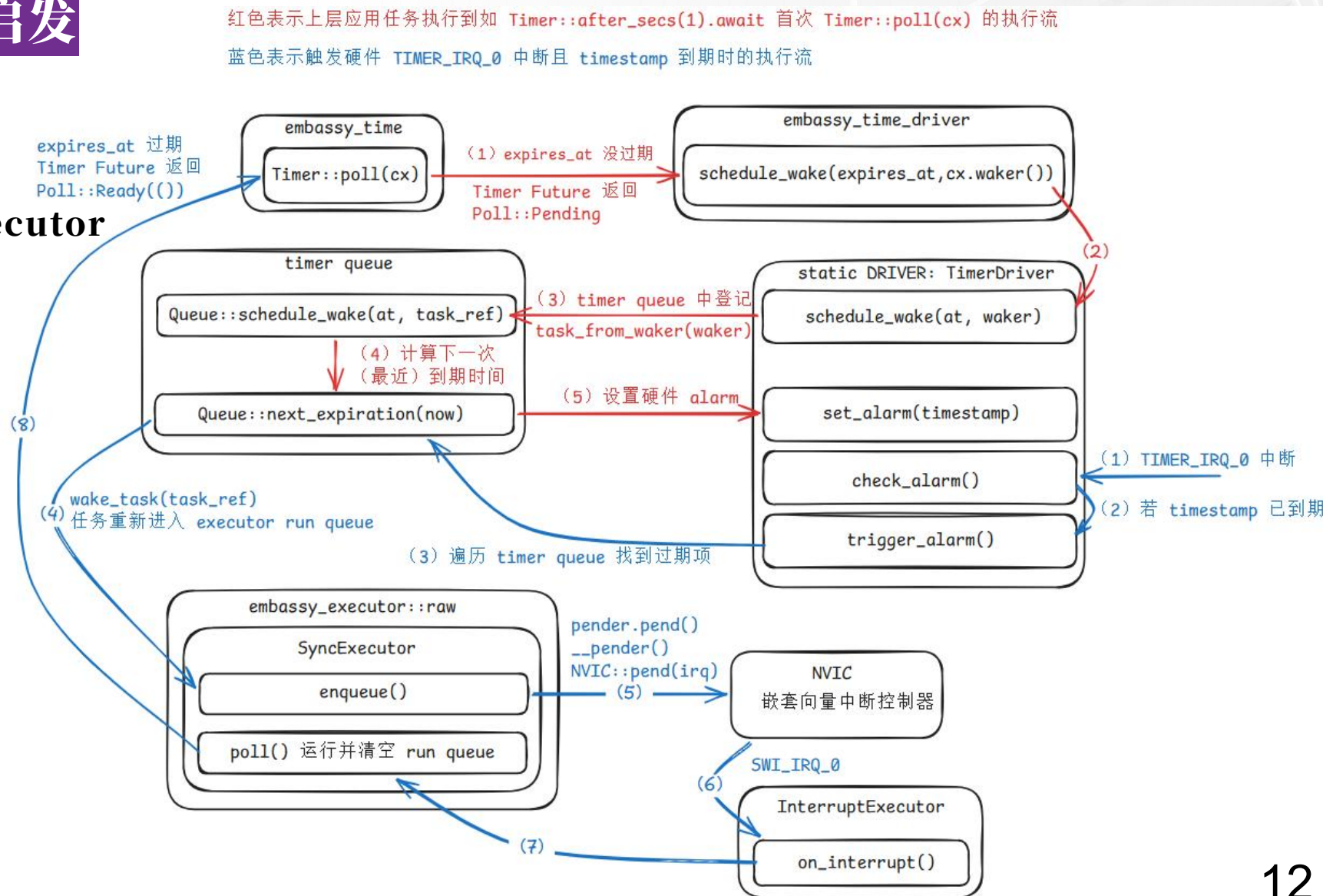
设计思路与取舍

Embassy 对设计的启发

2) embassy timer

2.1) embassy-rp

2.2) 基于中断的 executor



设计思路与取舍

Embassy 对设计的启发

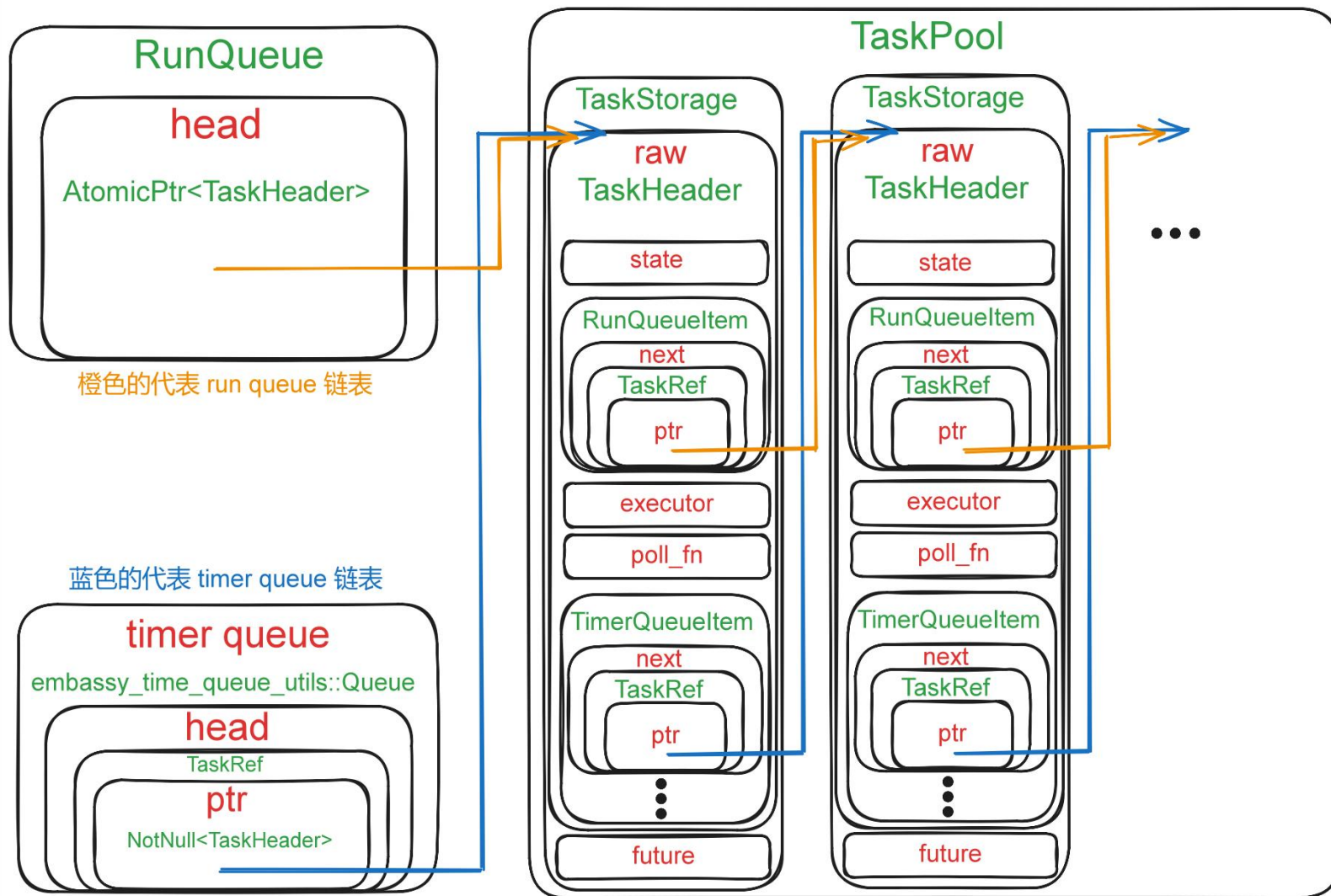
3) embassy 主要数据结构之间的转换

3.1) Waker 和 TaskRef 可以相互转换

3.2) TaskRef 存储一个指向 TaskHeader 的指针

3.3) TaskHeader 存储任务元信息，存储在 TaskStorage 中，是其第一个字段 raw

3.4) TaskPool<F,N> 是一组 N 个 TaskStorage<F>，每个 TaskStorage<F> 是一个可容纳 1 个任务实例的槽位



embassy 的双链表: timer queue 和 RunQueue

设计思路与取舍

设计思路

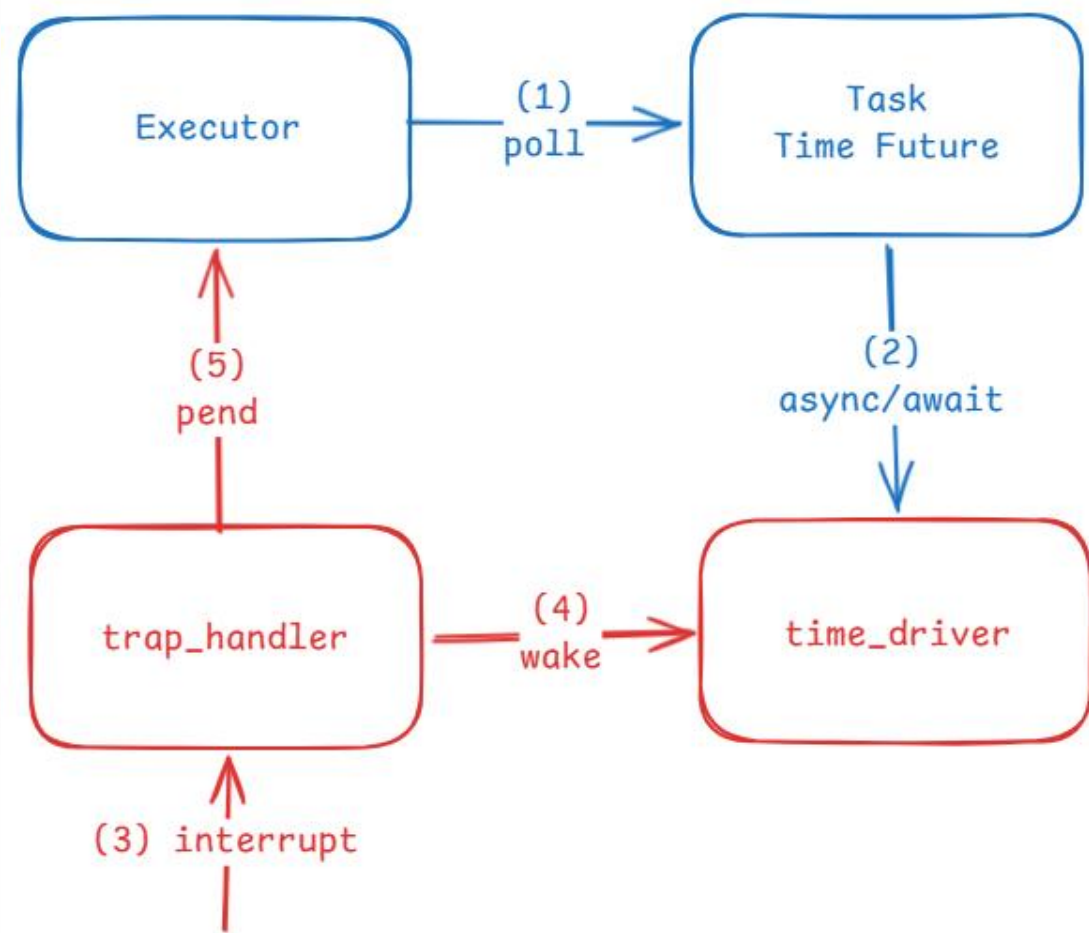
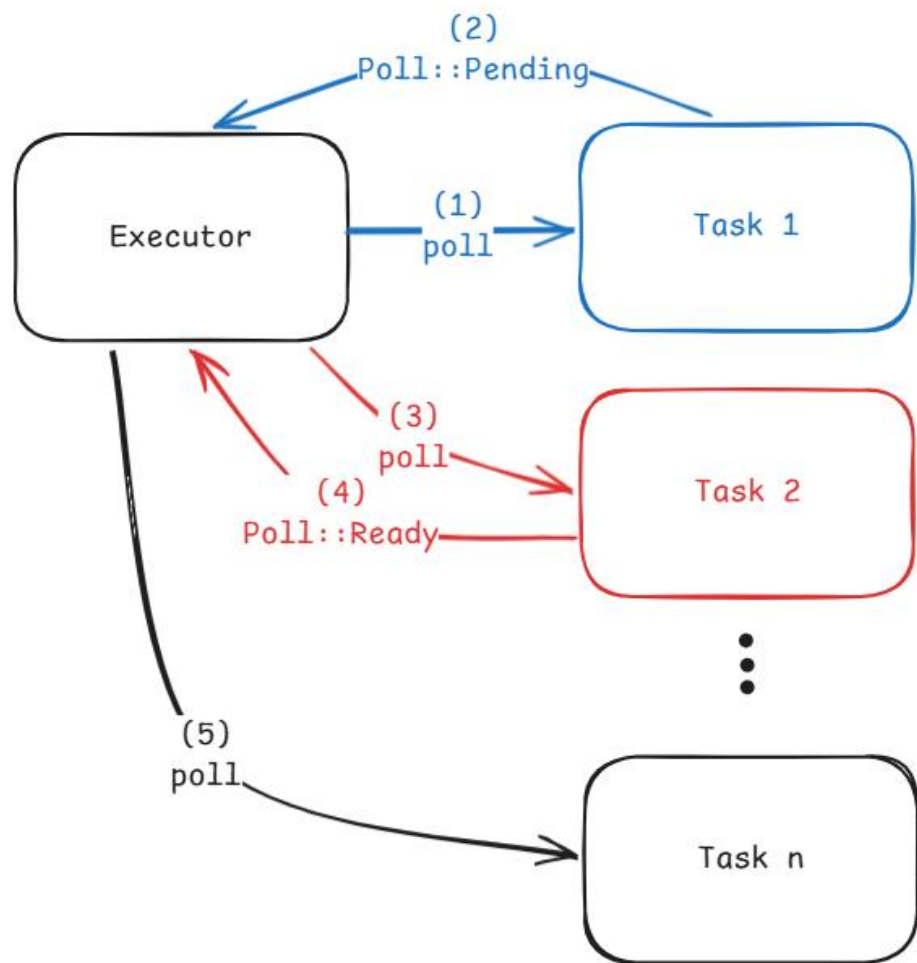
- 1) 重点实现和分析 OS tick, sys_sleep 相关 SupervisorTimer 路径
- 2) 参考 Embassy 模式
 - 2.1) 在 rCore-N 内核中引入 Timer Future, Waker, TimerDirver, Executor, timer queue

设计取舍

- 1) 不直接照搬 Embassy InterruptExecutor
 - 1.1) wake/poll 均位于 SupervisorTimer 中断上下文
- 2) 不保留独立 OS tick 设置路径, 和 sys_sleep 统一由 async_timer 管理
- 3) 不直接恢复传统 Blocked sys_sleep, 而是折中实现 suspend-loop sys_sleep
- 4) 使用标志位延后处理 OS tick Timer Future 回调

设计思路与取舍

设计思路



设计思路与取舍

设计取舍一：不直接照搬 Embassy InterruptExecutor

1) 初始设想：参考 Embassy 的两阶段模式

1.1) 在 S Timer 中断中完成到期检查与 Waker 唤醒，再通过 S Software Interrupt 执行 `executor poll()`

2) 问题：rCore-N 的 OS 时间片逻辑依赖 S Timer Trap 上下文

2.1) 如果把 `poll` 挪到 S Software Interrupt，时间片到期后的 `suspend_current_and_run_next()` 将不再运行在原有的 S Timer Trap 语义下

3) 解决：将 `wake` 与 `Executor::poll()` 统一放在 S Timer Trap 中执行

设计思路与取舍

设计取舍二：不保留独立 OS tick 路径

1) 备选设想：OS tick 和 sleep timer 独立

1.1) 保留原有 `set_next_trigger()` -> `sbi_set_timer()` 作为 OS tick, 同时只让 sleep timer 走 async timer

2) 问题：两套独立调用路径对硬件的竞争

2.1) 原有 timer interrupt 逻辑和 async timer 逻辑都会调用 `sbi_set_timer()`

2.2) 如果两套路径并存, 就会互相覆盖下一次硬件 timer 的触发时间

3) 解决：OS tick 也并入同一套 `async_timer` 时间管理路径

设计思路与取舍

设计取舍三：OS tick 统一后的问题

1) 问题一：Executor 重入

1.1) `after_ticks(...)` 创建 Timer Future Task 并将其加入 Executor 任务队列时会立刻 `Executor::poll()` 一次

1.2) 当 OS tick 也走 async timer 路径后，如果在 OS tick 超时回调中直接 `after_ticks(...)` 注册下一次 tick，会在 `Executor::poll()` 尚未退出时再次进入 `Executor::poll()`

2) 问题二：中途结束 `Executor::poll()`

2.1) 如果在 poll 过程中直接调用 `suspend_current_and_run_next()` 会中途切断执行器遍历任务队列

3) 解决：OS tick 超时回调逻辑延后

3.1) OS tick 到期时超时回调逻辑改成只设置 `PENDING_OS_TICK` 标志

3.2) 待 `Executor::poll()` 完成后，再由 S Timer 上下文中紧接着的 `handle_os_tick()` 视标志的值情况注册下一次 tick，并执行 `suspend_current_and_run_next()`

设计思路与取舍

设计取舍四：不直接恢复 Blocked 的 sys_sleep

1) 问题：中断上下文获取 ready_queue 锁导致内核 panic

1.1) rCore-N 运行在多 hart 环境，多个 hart 共享全局 ready_queue

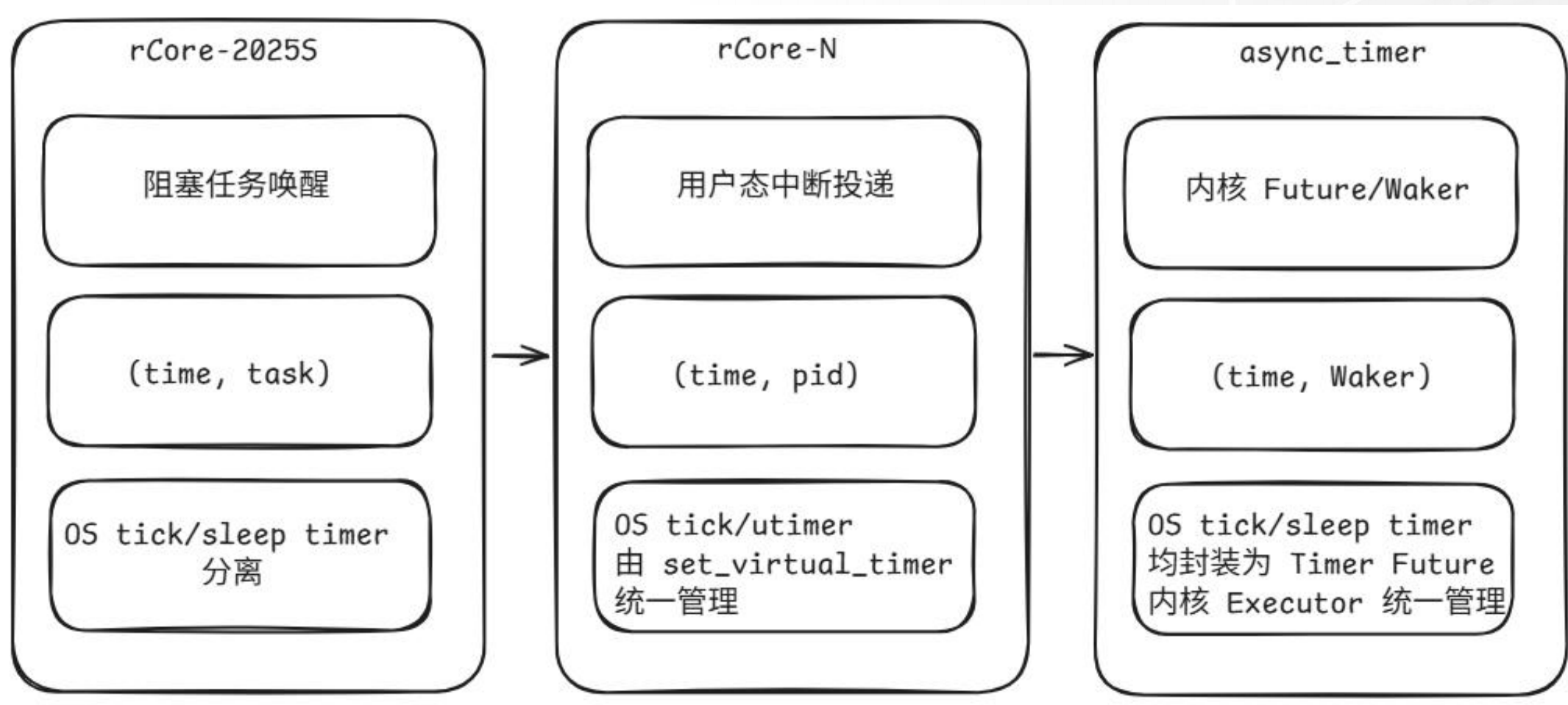
1.2) 若沿用传统 blocked sleep，任务调用 sleep 时需要移出 ready_queue，timer 到期后又要在中断上下文中重新入队

1.3) 这会引入共享 ready_queue 持锁、竞争以及调度一致性问题

2) 解决：sys_sleep 实现为 suspend-loop

2.1) timer 超时回调只修改 suspend-loop 退出条件，任务后续再次被调度时再返回

设计思路与取舍



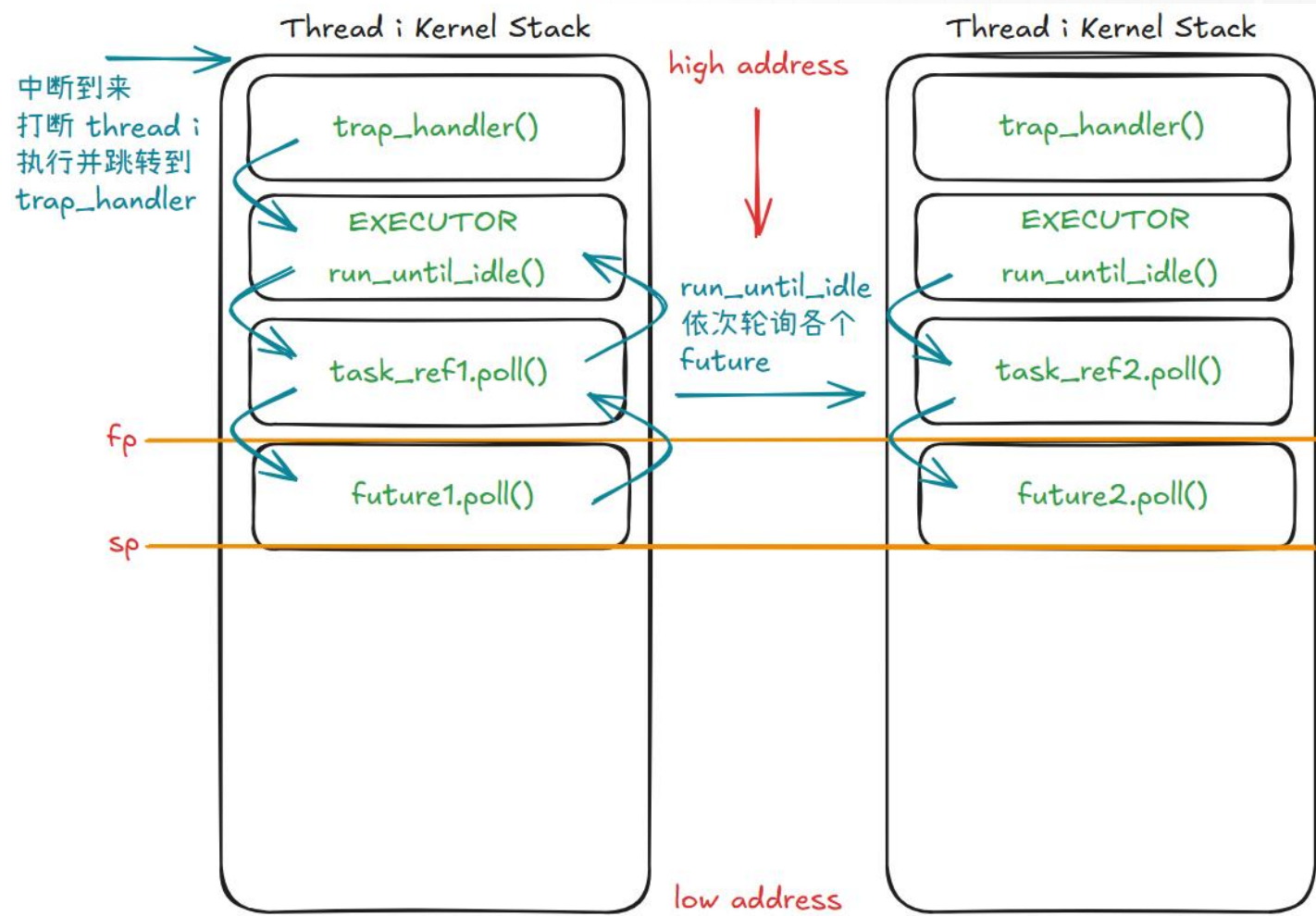


05.7

正确性与可用性

正确性与可用性

正确性：Rust Future 无栈协程栈帧分析



正确性与可用性

可用性：实验设计与运行演示

1) 实验背景介绍

- 1.1) 基于 `async_timer` 实现的 `sys_sleep` 提供给用户态的接口: `sleep_blocking` 函数
- 1.2) 设计不同的用户态程序作为测试用例, 调用 `sleep_blocking` 函数, 验证可用性

2) 实验设计

- 2.1) 基础响应误差测试
- 2.2) 并发 `sleep` 与 CPU load 压力测试
- 2.3) 不同到期顺序测试

3) 运行演示视频

正确性与可用性

可用性：基础响应误差测试

1) 实验内容

1.1) 验证单个进程调用 `sleep_blocking` 函数能否正常返回

1.2) 观察不同目标等待时间下的响应误差

2) 实验结果

目标时间/ms	1	2	3	4	5	6	7	8	9	10
10	12	12	12	12	12	11	11	11	12	12
50	53	52	51	52	51	52	52	52	51	53
100	102	102	101	102	101	102	102	101	102	102
500	502	501	501	501	502	501	502	501	501	502

正确性与可用性

可用性：并发 sleep 与 CPU load 压力测试

1) 实验内容

- 1.1) 验证多核调度下 sleep_blocking 是否仍然能正确完成定时等待
- 1.2) 同时跑多个进程调用 sleep_blocking 函数
- 1.3) 额外创建若干 CPU 密集型 cpu load 用户进程

2) 实验结果

CPU load 进程数	sleep 进程数	平均耗时 /ms	最小耗时 /ms	最大耗时 /ms	平均误差 /ms
0	10	102.2	101	104	2.2
1	10	101.5	101	103	1.5
4	10	103.2	101	107	3.2

「正确性与可用性

可用性：不同到期顺序测试

1) 实验内容

- 1.1) 验证异步 timer queue 是否能够按照不同定时事件的到期时间正确处理多个 sleep 请求
- 1.2) 在同一个用户态测例创建多个子进程分别以不同目标时间调用 sleep_blocking
- 1.3) 较短目标时间的子进程应整体上更早返回

2) 实验结果

2.1) 重复 10 次，顺序正确 10 次，异常 0 次。以下是其中一次的结果：

创建顺序	进程编号	目标时间 /ms	开始时间 /ms	结束时间 /ms	实际经过 /ms	响应误差 /ms	按结束时间排序后返回顺序
1	pid 2	100	5206	5306	100	0	4
2	pid 3	10	5216	5232	16	6	1
3	pid 4	50	5224	5274	50	0	3
4	pid 5	20	5234	5255	21	1	2

「正确性与可用性

可用性：运行演示视频





06.7

结论

结论

1) 在 rCore-N 中实现了一个基于 Rust Future/Waker 的异步 timer 原型

2) 完成了从 Future 挂起、Waker 注册、timer 到期、中断唤醒到

Executor::poll() 的完整链路

3) 在多核环境下, sleep_blocking 能够稳定工作, 并保持可解释范围内的响应误差

4) 结果表明: Rust 异步机制接入操作系统内核 timer 路径是可行的



07.7

后续工作设想

后续工作设想

- 1) 当前 `sys_sleep` 仍是 `suspend-loop` 折中实现，不是真正的 `blocked sleep`
- 2) `timer queue` 目前基于动态 `Vec`，且因此大规模并发下存在线性扫描开销
- 3) 异步任务生命周期管理、跨 `hart wake` 与迁移、`poll` 上下文拆分仍需完善
- 4) 后续重点是 `deferred wake`、真正 `blocked sleep`、`timer` 数据结构优化以及异步驱动推广到串口/块设备/网络等场景

The background of the slide features a faded image of the Tsinghua University gate and the university's name in Chinese characters (清华大学) on a stone wall. The right side of the slide is a solid purple color.

谢谢 请各位老师批评指正

汇报人：黄昊颖 指导老师：向勇